

```
function controlFlow() { ... }
```

```
if (condition) { return true; }
```



Solidity智能合约开发

第4.1课：控制流语句

掌握条件语句、循环结构、错误处理与Gas优化的最佳实践



```
require(condition, "Error");
```

讲师 [Layer]

```
for (let i = 0; i < n; i++) { ... }
```

今天的学习内容



控制流概述

理解程序执行的三种基本结构（顺序、选择、循环），以及在智能合约中的应用。



条件语句

掌握 `if`、`if-else`、`else if` 等逻辑判断结构。



循环语句

使用 `for`、`while` 等循环结构实现重复逻辑。



Gas成本

分析循环与条件语句对Gas消耗的影响，掌握安全循环实践方法。



错误处理

掌握 `require`、`assert`、`revert` 等机制的用法。



实战案例

通过投票系统和状态机代码示例，理解控制流在智能合约中的应用。

什么是控制流？



顺序结构

代码从上到下依次执行，这是最简单的执行路径。

```
function sequential() {  
  statement1();  
  statement2();  
  statement3();  
}
```



选择结构

程序根据条件判断，选择执行不同的代码分支。

```
if (condition) {  
  // 条件为真  
} else {  
  // 条件为假  
}
```



循环结构

程序重复执行某个代码块，直到满足特定终止条件。

```
for (init; condition; update) {  
  // 循环体  
}
```



Solidity中的特殊考量



Gas成本

循环次数越多，消耗的Gas成本越高，可能导致交易失败。



无限循环风险

无限循环会导致Gas耗尽，交易回滚，并可能造成资金损失。

条件语句：if与if-else

</> if语句基础

if语句是最基本的条件判断结构，用于在指定条件为 **true** 时执行一段代码。

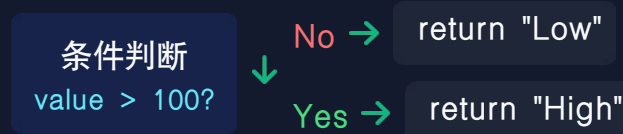
```
function checkAge(uint age)
    public pure returns (bool)
{
    if (age >= 18) {
        return true;
    }
    return false;
}
```

if-else语句

if-else提供了二选一的逻辑分支，当if条件不满足时，程序将执行else块中的代码。

```
function checkValue(uint value)
    public pure returns (string memory)
{
    if (value > 100) {
```

🔗 执行流程图



💡 关键点

- ✅ 二选一的逻辑：条件为 **true** 执行if块，条件为 **false** 执行else块。
- ✅ if或else块中的代码必定会执行其中一个。
- ✅ 条件表达式必须是布尔值，例如：age >= 18、balance > 0、msg.sender == owner

多条件判断与三元运算符

else if 链

用于处理多个互斥条件的多分支逻辑，程序从上到下依次检查条件，第一个满足的条件对应的代码块会被执行。

```
function getGrade(uint score) public pure returns (string memory) {
    if (score >= 90) {
        return "A";
    } else if (score >= 80) {
        return "B";
    } else if (score >= 70) {
        return "C";
    } else if (score >= 60) {
        return "D";
    } else {
        return "F";
    }
}
```

⚠️ 注意事项： 条件顺序很重要！第一个满足的条件会被执行，后续条件不再检查。

三元运算符

提供了一种简洁的条件赋值方式，适用于简单的二选一逻辑。

```
// 语法：条件 ? 值1 : 值2
function max(uint a, uint b) public pure returns (uint) {
    return a > b ? a : b;
}
```

适用场景对比

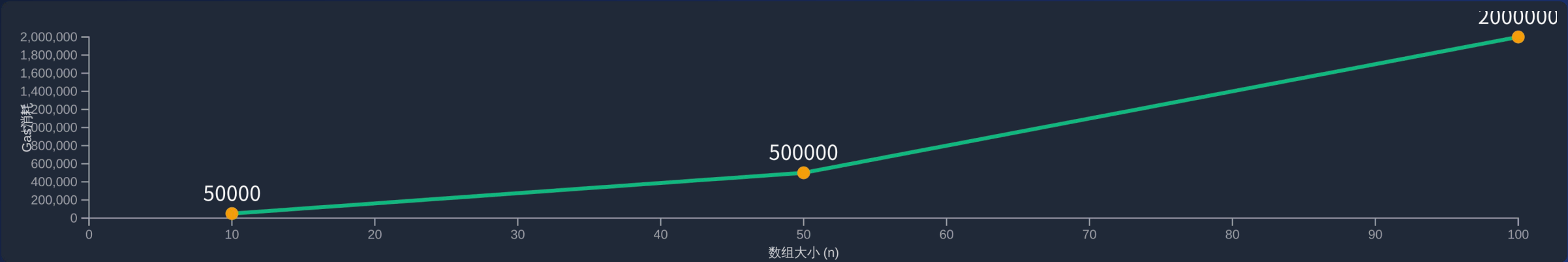
使用方式	适合场景	不适合场景
if-else	复杂多重判断 多行代码执行 需要注释说明	简单二选一 单行简单赋值 追求极简代码
三元运算符	简单二选一 单行条件赋值 追求代码简洁	复杂多重判断 多行代码执行 需要嵌套判断

循环的Gas成本与安全实践

⚠️ 危险示例：无限制循环

当数组元素过多时，Gas消耗会线性增长，可能触及区块Gas限制，导致交易失败。

Gas消耗随数组大小增长图表：



限制循环次数

使用require限制数组长度，避免过多迭代。



使用Mapping

利用mapping实现O(1)查询，避免循环遍历。



分批处理

每次处理少量数据，分多次交易完成，降低单次Gas消耗。



链下计算

前端计算后只提交结果到链上，节省Gas消耗。

错误处理：require、assert与revert

require

用于 外部输入验证 和 前置条件检查

```
require(balance >= amount, "余额不足");
```

- > 验证用户输入
- > 检查合约状态
- > 支持错误消息

assert

用于 内部检查 和 不变量验证

```
assert(totalSupply == balanceBefore);
```

- > 检查内部条件
- > 确保不变量
- > 不支持错误消息

revert

用于 灵活错误处理 和 自定义错误

```
revert("操作失败");  
revert CustomError();
```

- > 复杂条件检查
- > 自定义错误类型
- > 高效错误消息

↔ require 与 assert 的关键区别

require - 外部验证

- ✓ 用于验证用户输入
- ✓ 失败原因是用户错误
- ✓ 支持自定义错误消息

assert - 内部检查

- ✓ 用于检查内部条件
- ✓ 失败原因是代码bug
- ✓ 不支持错误消息字符串

错误处理：require/assert/revert

特性	✅ require	🚫 assert	⚠️ revert
用途	外部验证 / 输入验证	内部检查 / 不变量验证	灵活控制 / 自定义错误
失败原因	用户错误	代码bug	业务逻辑错误
错误消息	支持字符串消息	不支持	支持字符串和自定义错误
Gas返还	是	是 (0.8.0+)	是
使用频率	很高	低	中等

✅ require 示例

```
function transfer(address to, uint256 amount) {
    require(to != address(0), "无效地址");
    require(amount > 0, "金额必须大于0");
    require(balanceOf[msg.sender] >= amount, "余额不足");

    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;
}
```

适用于用户输入验证、权限检查、状态检查

🚫 assert 示例

```
function transferWithCheck(address to, uint256 amount) {
    require(to != address(0), "无效地址");

    uint256 oldBalance = balanceOf[msg.sender];
    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;

    assert(balanceOf[msg.sender] == oldBalance - amount);
}
```

适用于检查不变量、调试断言、内部状态一致性

⚠️ 自定义错误示例

```
error InsufficientBalance(uint256 balance, uint256 needed);

function transfer(address to, uint256 amount) {
    if (balanceOf[msg.sender] < amount)
        revert InsufficientBalance(balanceOf[msg.sender], amount);

    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;
}
```

自定义错误节省Gas，支持参数，类型安全

自定义错误与最佳实践

❗ 自定义错误

定义与使用

```
error InsufficientBalance( uint requested, uint available);  
error InvalidAddress( address addr);
```

```
function transfer( address to, uint amount) public {  
    if (to == address (0)) {  
        revert InvalidAddress(to);  
    }  
    if (balances[msg.sender] < amount) {  
        revert InsufficientBalance(  
            amount,  
            balances[msg.sender]  
        );  
    }  
}
```

自定义错误 vs 字符串错误

- ✓ **Gas消耗:** 自定义错误 ~节省50%
- ✓ **参数支持:** 自定义错误 支持
- ✓ **类型安全:** 自定义错误 提供

⚙️ 错误处理最佳实践



尽早检查

在函数开始处进行参数验证，避免后续复杂逻辑



清晰消息

提供明确的错误信息，便于调试和用户理解



优化检查顺序

先检查低成本条件，再检查高成本条件



防御性编程

假设外部输入不可信，始终进行验证



减少嵌套

使用早期返回减少if-else嵌套层数



一致性

在整个项目中保持一致的错误处理风格

开发技巧与常见陷阱



实用技巧



条件判断技巧

使用early return、modifier、避免深层嵌套、三元运算符简化代码



循环使用技巧

优先考虑mapping、严格限制循环次数、避免嵌套循环、链下计算、分批处理



错误处理技巧

验证外部输入、尽早检查、清晰错误消息、自定义错误提高Gas效率



安全编程技巧

防御性编程、检查返回值、Checks-Effects-Interactions模式、ReentrancyGuard



常见陷阱



无限循环

忘记更新循环变量导致Gas耗尽，如：for(i=0; i<n; i--)



循环中修改数组

边遍历边删除数组元素可能导致跳过元素，应从后往前遍历



整数溢出

0.8.0之前版本整数溢出不会报错，可能导致意外行为，推荐使用SafeMath



重入攻击

先转账后更新状态导致多次调用，应采用Checks-Effects-Interactions模式



block.timestamp操纵

不要将block.timestamp用于关键随机性，推荐使用Chainlink VRF

实战案例：投票系统与状态机模式

投票系统合约

通过投票合约展示多种控制流应用

```
contract VotingSystem {
  struct Proposal {
    bytes32 description;
    uint voteCount;
  }

  mapping(uint => Proposal) proposals;
  mapping(address => bool) hasVoted;

  function createProposal(bytes32 _description) public {
    require(proposals.length < 10, "提案数量已达上限");
    proposals.push(Proposal(_description, 0));
  }

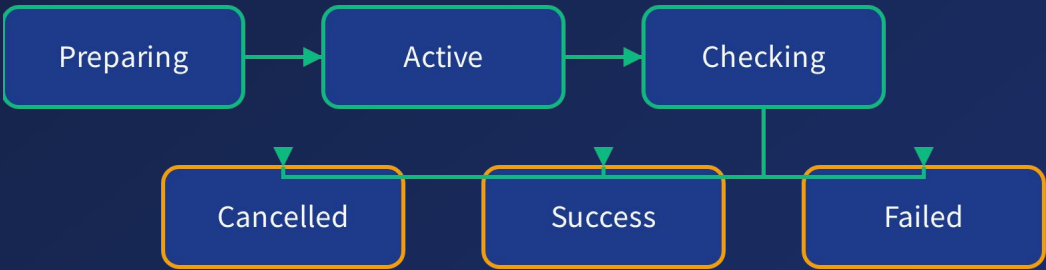
  function vote(uint _proposalIndex) public {
    require(_proposalIndex < proposals.length, "提案不存在");
    require(hasVoted[msg.sender] == false, "已投票");
    hasVoted[msg.sender] = true;
    proposals[_proposalIndex].voteCount++;
  }

  function getWinner() public view returns(uint) {
    uint maxVotes = 0;
    uint winnerIndex = 0;

    for(uint i = 0; i < proposals.length; i++) {
      if (proposals[i].voteCount > maxVotes) {
        maxVotes = proposals[i].voteCount;
        winnerIndex = i;
      }
    }
  }
}
```

状态机模式

通过状态机管理复杂业务逻辑



```
contract StateMachine {
  enum State { Preparing, Active, Checking, Success, Failed, Cancelled }

  modifier inState(State expected) {
    require(currentState == expected, "状态不正确");
    _;
  }

  function start() public inState(State.Preparing) {
    currentState = State.Active;
  }

  function contribute() public inState(State.Active) {
    // 处理贡献逻辑
  }
}
```

课程总结



控制流的基础地位

控制流语句是构建智能合约逻辑的基础，掌握 `if-else` 、 `for` 、 `while` 等语句的正确使用方法，是编写复杂业务逻辑合约的前提条件。



Gas成本控制的重要性

在区块链上执行操作需要消耗Gas，不当的循环控制和数据处理方式可能导致Gas耗尽、交易失败甚至资金损失。始终将Gas优化放在开发流程的重要位置。



安全开发原则

使用 `require` 进行输入验证，`assert` 检查内部状态，`revert` 处理自定义错误，遵循"先检查，后执行"的原则，避免常见陷阱如无限循环和重入攻击。



继续学习：探索特殊类型与全局变量

下节预告：特殊类型与全局变量



address类型

深入理解以太坊地址类型及其特性

- 地址类型的定义与特点
- 地址的成员方法： `transfer` 和 `send`
- 地址类型的安全使用实践



全局变量

掌握Solidity中关键的全局变量及其应用

- `msg` : 交易信息 (sender, value, data)
- `block` : 区块属性 (timestamp, number)
- `tx` : 交易属性与上下文



enum枚举

学习枚举类型在智能合约中的应用

- 枚举的定义与使用方法
- 状态机模式中的枚举应用
- 枚举类型与可读性、安全性的关系



constant/immutable优化

了解常量和不可变量如何优化Gas消耗

- `constant` : 编译时常量
- `immutable` : 部署时固定值
- Gas优化效果与最佳实践